

## **Project 1: Movie Recommender System**

### **Movie Recommender System using NLP & Cosine Similarity**

#### **1. Introduction**

A Movie Recommender System is an intelligent application that suggests movies based on user preferences. This project uses **Natural Language Processing (NLP)** and **machine learning techniques** to recommend movies similar to a selected title.

The system analyzes movie metadata such as genres, keywords, cast, and crew to find similarities between movies. It is widely used in platforms like Netflix and Amazon Prime to enhance user experience.

#### **2. Objective**

- To build a system that recommends movies based on similarity
- To implement content-based filtering using NLP techniques
- To understand vectorization and similarity metrics
- To develop an interactive web application for real-time recommendations

#### **3. Dataset**

- A movie dataset containing:
  - o Movie title
  - o Movie ID
  - o Genres
  - o Keywords
  - o Cast
  - o Crew
- Data preprocessing includes:
  - o Combining multiple columns into a single feature (tags)
  - o Converting text to lowercase
  - o Removing unnecessary spaces
- Processed data is stored using:
  - o movies\_eve.pkl
  - o similarity\_eve.pkl

## 4. Methodology

### Step 1: Data Preprocessing

- Merge relevant features into a single column (tags)
- Clean and normalize text data

### Step 2: Feature Extraction

- Use **CountVectorizer**
- Convert text into numerical vectors
- Limit features to 5000 and remove stop words

### Step 3: Similarity Computation

- Apply **Cosine Similarity**
- Compute similarity between all movies

### Step 4: Model Saving

- Save processed data and similarity matrix using pickle
- Files used:
  - o movies\_eve.pkl
  - o similarity\_eve.pkl

### Step 5: Deployment

- Build web app using **Streamlit**
- User selects a movie → system recommends similar movies
- Integrate **TMDB API** to fetch movie posters

## 5. Tools & Technologies

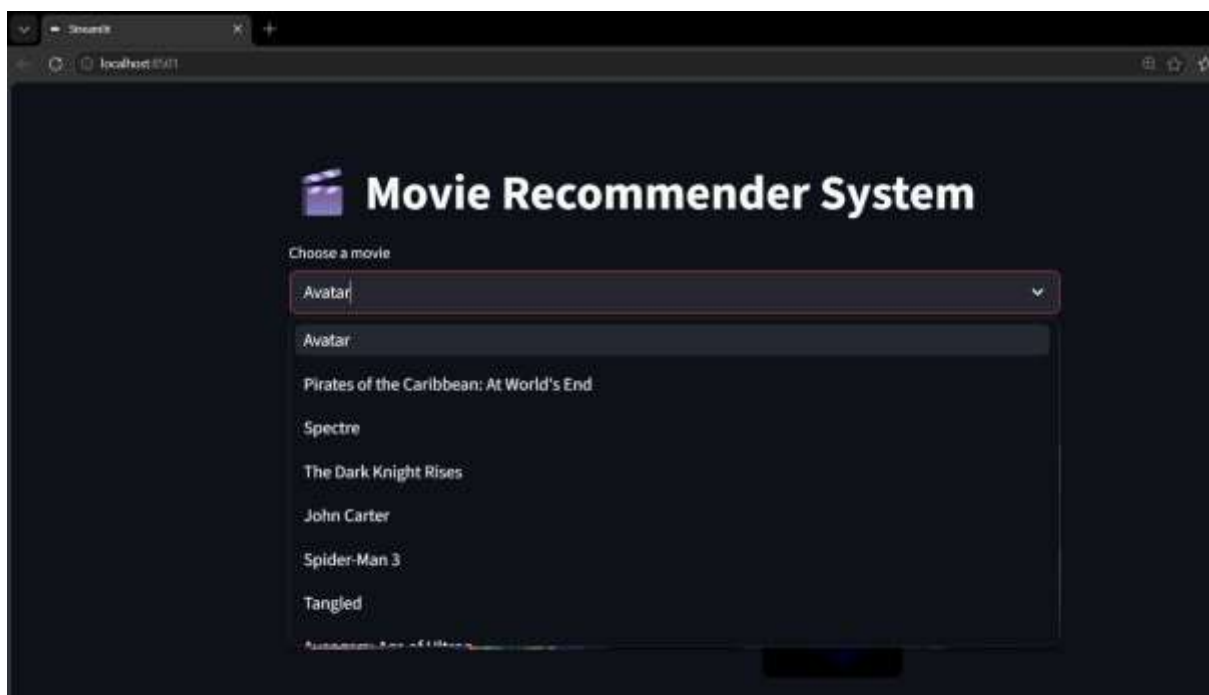
- Programming Language: Python
- Libraries: Pandas, NumPy, Scikit-learn, Pickle, Requests
- Framework: Streamlit
- API: TMDB (The Movie Database)
- Environment: Jupyter Notebook & VS Code

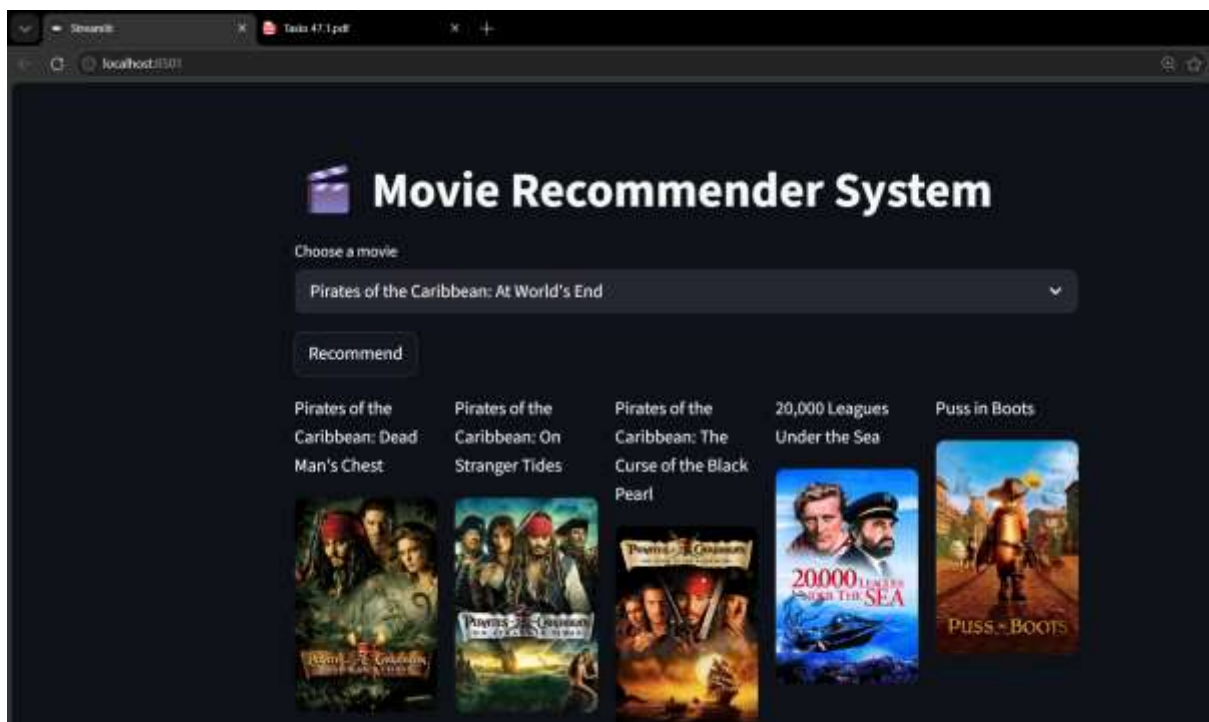
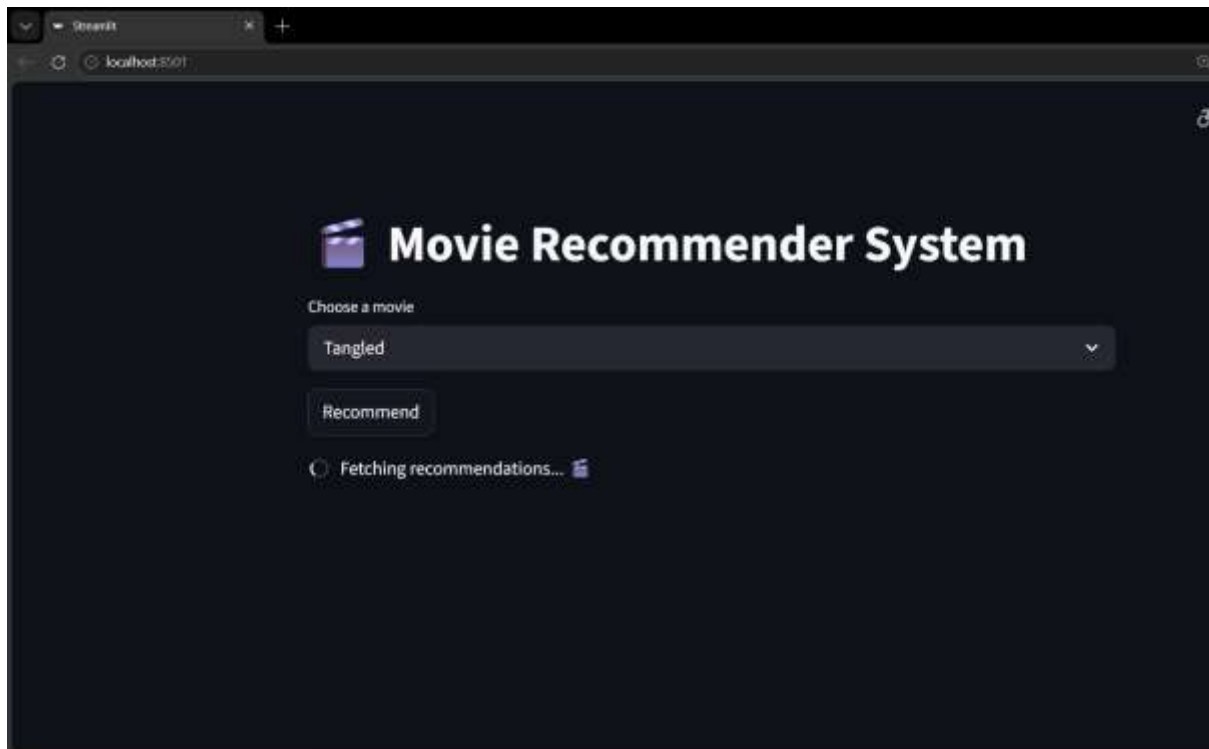
## 6. Results

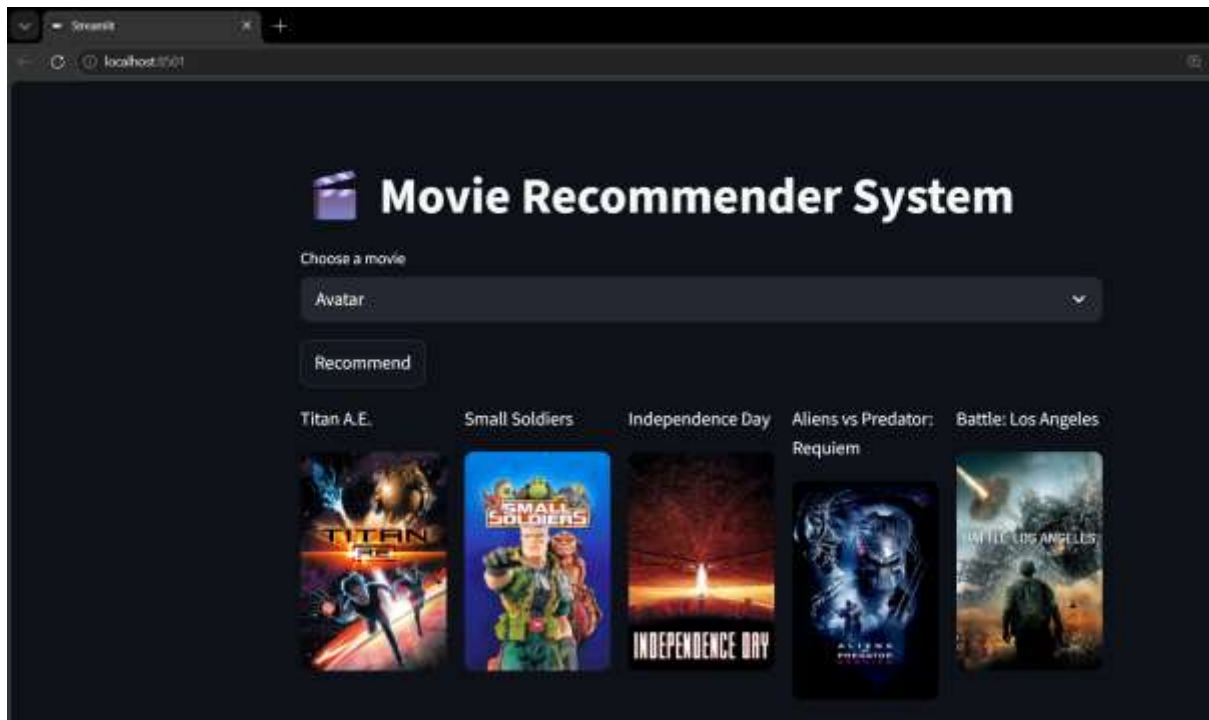
- The system successfully recommends similar movies
- Cosine similarity provides accurate recommendations
- Posters are fetched dynamically using TMDB API
- The application responds in real-time with optimized performance

### OUTPUT:

- Displays top 5 recommended movies
- Shows movie posters along with titles
- Interactive user interface using Streamlit







## 8. Applications

- OTT platforms (Netflix, Amazon Prime)
- Movie discovery systems
- Personalized recommendation engines
- Entertainment applications

## 9. Conclusion

This project demonstrates how NLP and machine learning techniques can be used to build an efficient recommendation system. The use of CountVectorizer and cosine similarity enables accurate similarity detection, while Streamlit provides an interactive interface.

The system can be further enhanced by integrating collaborative filtering and deep learning techniques to improve personalization and scalability.

## **Project 2: Book Recommendation System**

### **Book Recommendation System using Content-Based Filtering**

#### **1. Introduction**

A Book Recommendation System is an intelligent application that suggests books based on user preferences. This project uses **Natural Language Processing (NLP)** techniques and **machine learning algorithms** to recommend books similar to a selected title.

The system analyzes textual features such as title, author, categories, and description to identify similarities between books. Such systems are widely used in platforms like Amazon Kindle and Goodreads to improve user experience.

#### **2. Objective**

- To build a system that recommends books based on similarity
- To implement content-based filtering using NLP techniques
- To understand TF-IDF vectorization and similarity metrics
- To develop a real-time interactive application using Streamlit

#### **3. Dataset**

- A book dataset containing:
  - o Book title
  - o Authors
  - o Categories
  - o Description
  - o Thumbnail (image URL)
- Data preprocessing includes:
  - o Combining title, authors, categories, and description into a single feature (tags)
  - o Converting text to lowercase
- Processed data is stored using:
  - o books.pkl
  - o similarity\_books.pkl

## 4. Methodology

### Step 1: Data Preprocessing

- Combine multiple columns into a single column (tags)
- Convert all text to lowercase for uniformity

### Step 2: Feature Extraction

- Use **TF-IDF Vectorizer**
- Convert text into numerical vectors
- Limit features to 8000 and remove stop words

### Step 3: Similarity Computation

- Apply **Cosine Similarity**
- Compute similarity scores between all books

### Step 4: Model Saving

- Save processed data and similarity matrix using pickle
- Files used:
  - o books.pkl
  - o similarity\_books.pkl

### Step 5: Deployment

- Build web application using **Streamlit**
- User selects a book → system recommends similar books
- Display book thumbnails along with titles

## 5. Tools & Technologies

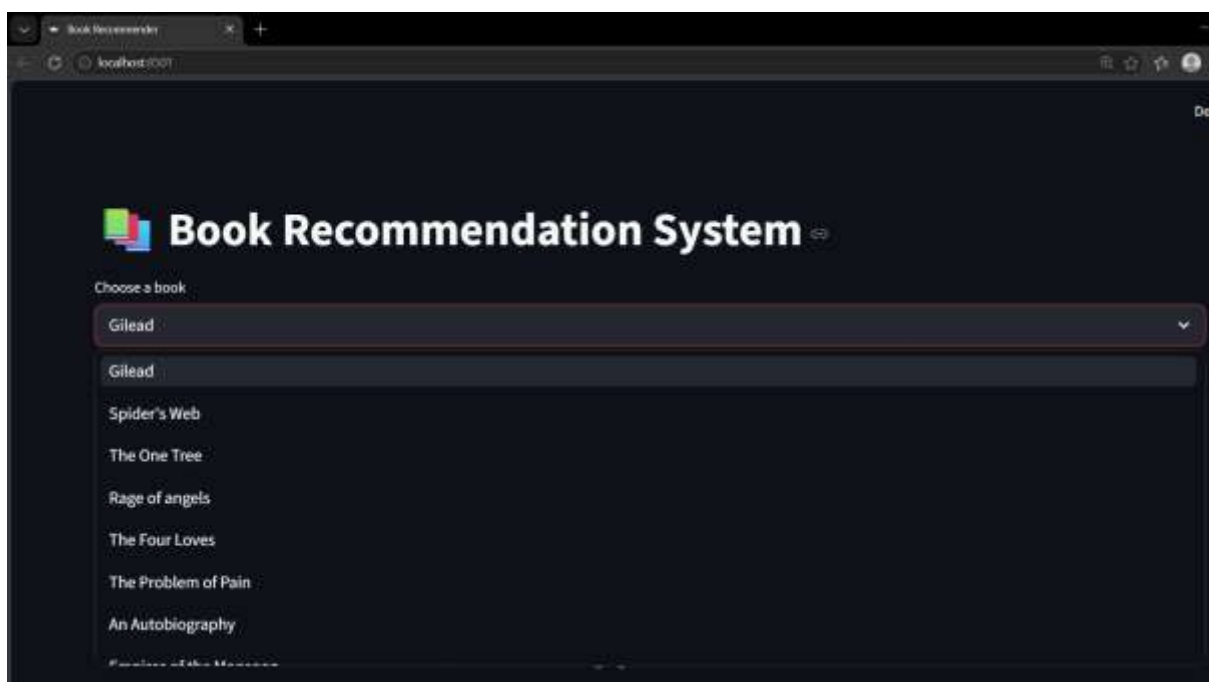
- Programming Language: Python
- Libraries: Pandas, NumPy, Scikit-learn, Pickle
- Framework: Streamlit
- Environment: Jupyter Notebook & VS Code

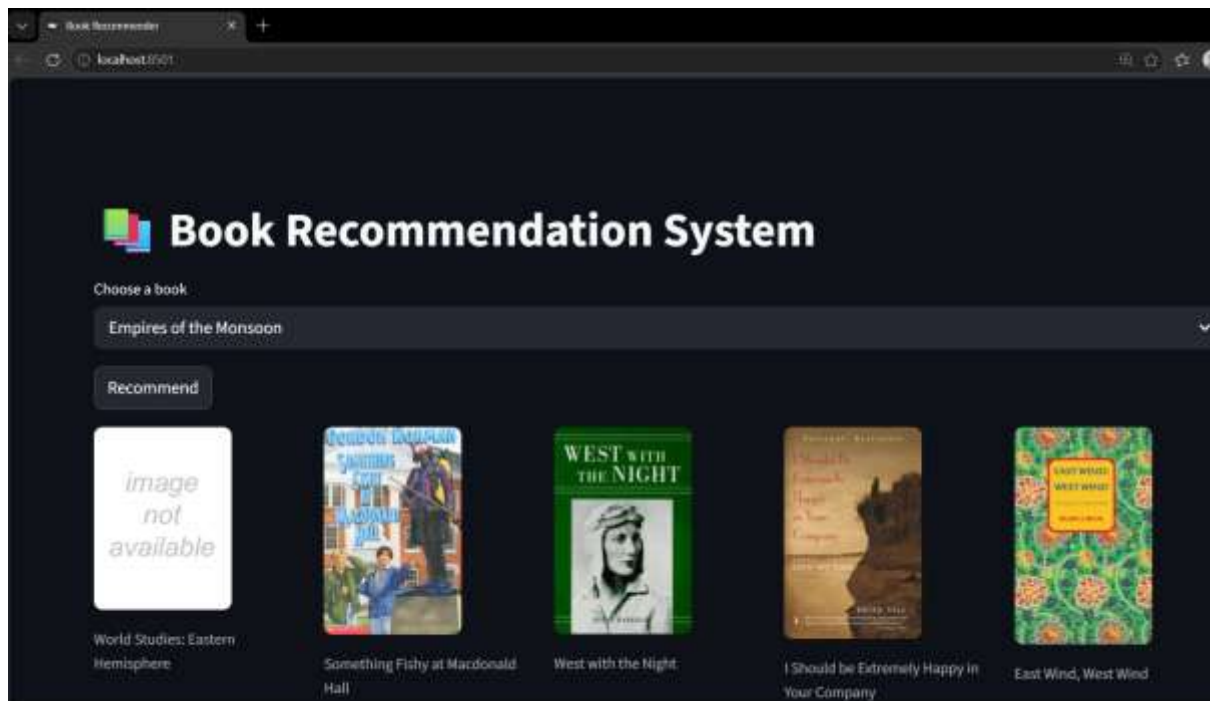
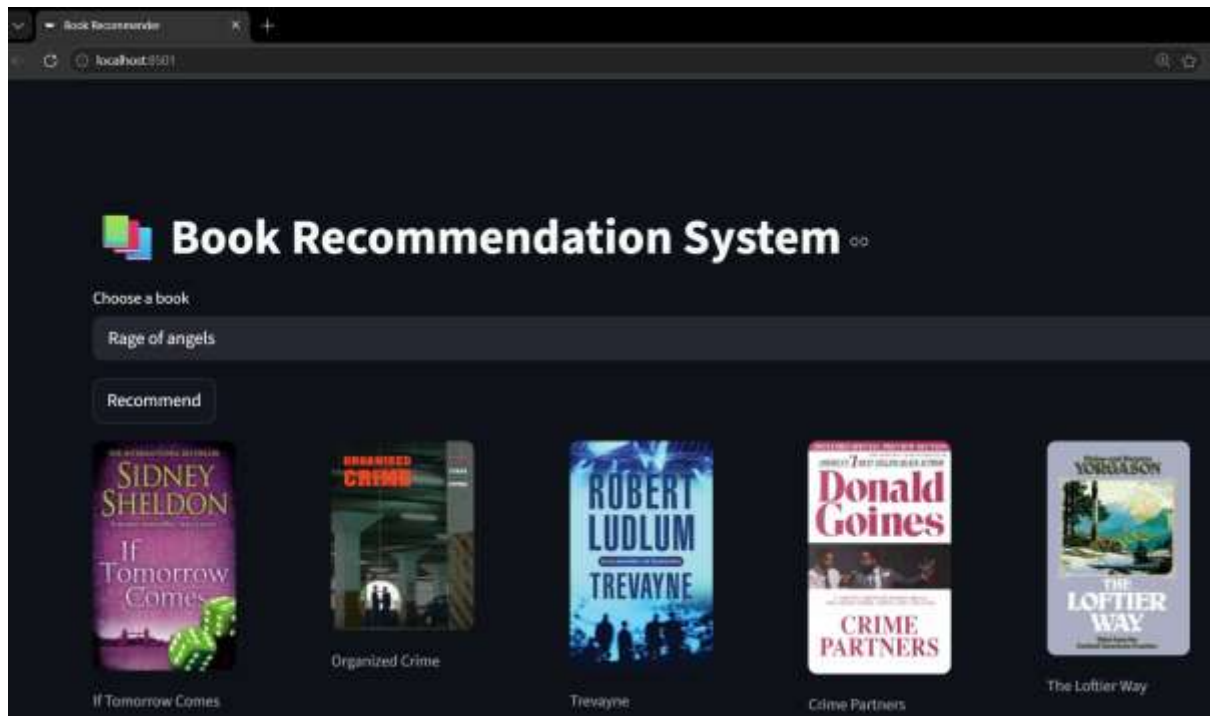
## 7. Results

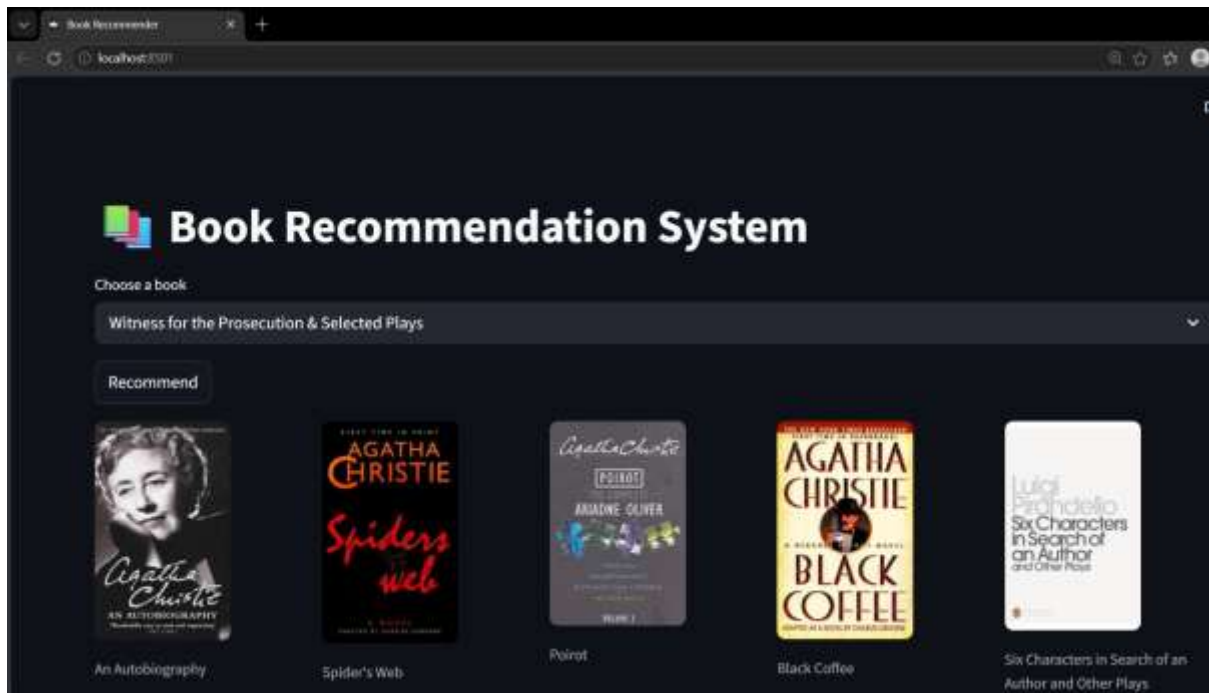
- The system successfully recommends similar books
- TF-IDF improves relevance by giving importance to significant words
- Cosine similarity provides accurate similarity matching
- The application delivers real-time recommendations with a user-friendly interface

### OUTPUT:

- Displays top 5 recommended books
- Shows book thumbnails along with titles
- Interactive UI with selection dropdown







## 8. Applications

- Online book platforms (Amazon, Goodreads)
- Personalized reading recommendation systems
- Digital libraries
- E-learning platforms

## 9. Conclusion

This project demonstrates how NLP and machine learning techniques can be applied to build an effective book recommendation system. The use of TF-IDF vectorization enhances text representation, while cosine similarity ensures accurate recommendations.

The system can be further improved by integrating collaborative filtering, user preferences, and deep learning techniques to provide more personalized recommendations.